University of Nottingham
UK | CHINA | MALAYSIA

School of Computer Science

A Level 1 Module COMP1036 Autumn 2024/25

COMPUTER FUNDAMENTALS

Instructions:

Time allowed: One Hour

Answer ALL questions

Candidates may complete the front cover of their answer book and sign their desk card. **DO NOT turn the examination paper over until instructed to do so.**

Permitted resources:

Those whose first language is not English may use a standard translation dictionary to translate between that language and English provided that neither language is the subject of this examination.

No calculators are permitted in this examination.

Prohibited resources:

All other dictionaries, including subject specific translation dictionaries and electronic dictionaries. Electronic devices capable of storing and retrieving text.

Appended material: APPENDIX 1 A-instruction specification, APPENDIX 2 C-instruction specification

Additional material: None

Information for Invigilators:

Exam Papers must not be removed from the examination venue

QUESTION 1 (25 Marks)

(a) Convert the following 8-bit two's complement binary into decimal.
$a = 1011\ 0100_2$ $b = 0010\ 1111_2$ $c = 1111\ 1111_2$ (3 marks)

(b) Convert the following decimal values to 8-bit two's complement binary.
$a = -113$ $b = -87$ $c = 53$ (3 marks)

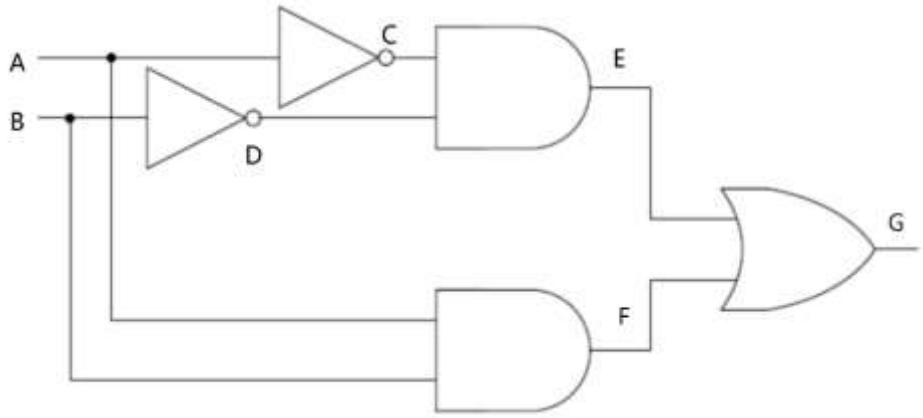
(c) Simplify the following Boolean Expression.

(i) $B(BD + \overline{A}D) + \overline{DA}$

(ii) $(AB + \overline{A}C)A + \overline{C}C$

(iii) $B(AC + A(AC)) + \overline{BB}$ (6 marks)

(d) Complete the truth table for the below composite gate. (8 marks)



(e) List five combinational chips and describe their functional features. (5 marks)

QUESTION 2 (25 Marks)

(a) This question is based on the symbolic assembly code below:

```
(START)
    @R2
    M = 0
    @R3
    M = 0
(LOOP)
    @R0
    D = M
    @R1
    D = D - M
    @IF_STATEMENT
    D; JLT
    @R1
    D = M
    @R0
    M = M - D
    @R2
    M = M + 1
    @LOOP
    0; JMP
(IF_STATEMENT)
    @R0
    D = M
    @END
    D; JNE
    @R3
    M = 1
(END)
    @END
    0; JMP
```

(i) Given M[0] = 2 and M[1] = 1. Derive the value of M[2] and M[3] after the execution of this piece of code. (2 marks)

(ii) Write an equivalent high-level pseudocode for the symbolic assembly code in (a). (4 marks)

(iii) Convert the last two lines of the symbolic assembly code in (a) to binary machine code:

@END
0; JMP

You may refer to APPENDIX 1 and 2 for this conversion. (2 marks)

(b) Draw a picture to show the final stack status after executing the VM code below:

(7 marks)

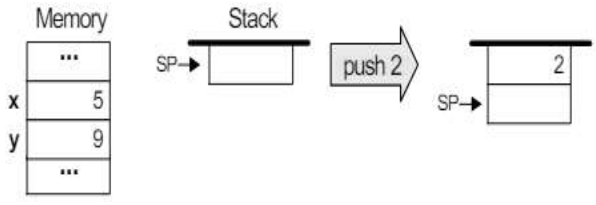
```
push constant 15
push constant 15
eq
push constant 10
push constant 11
eq
push constant 92
push constant 91
lt
push constant 91
push constant 92
lt
push constant 66
push constant 67
gt
push constant 66
push constant 66
gt
push constant 57
push constant 31
push constant 53
add
push constant 112
sub
neg
and
push constant 8
or
not
```

(c) Draw a diagram to show the stack operations, step by step, for the evaluation of the following expression:

```
d = (2 – x) * (y + 5)
```

The diagram of the initial memory state and the first stack operation are shown below. Remember to include the final memory state at the end of the stack operations.

(4 marks)



(d) Translate the following VM code into Hack symbolic assembly code:

```
push constant 1
add
```

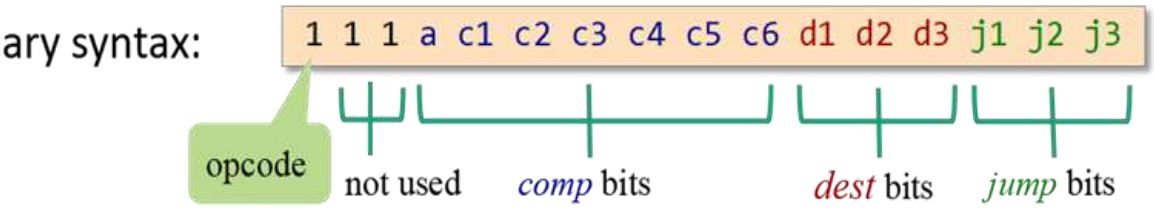
(6 marks)

APPENDIX 1 A-instruction specification
Symbolic syntax:
@value (Example: @21)
Binary syntax:
0value (Example: 0000000000010101)

APPENDIX 2 C-instruction specification

Symbolic syntax: *dest* = *comp* ; *jump*

Binary syntax:



comp		c1	c2	c3	c4	c5	c6
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D|A	D|M	0	1	0	1	0	1
a==0	a==1						

dest	d1	d2	d3	effect: the value is stored in:
null	0	0	0	The value is not stored
M	0	0	1	RAM[A]
D	0	1	0	D register
MD	0	1	1	RAM[A] and D register
A	1	0	0	A register
AM	1	0	1	A register and RAM[A]
AD	1	1	0	A register and D register
AMD	1	1	1	A register, RAM[A], and D register

jump	j1	j2	j3	effect:
null	0	0	0	no jump
JGT	0	0	1	if out > 0 jump
JEQ	0	1	0	if out = 0 jump
JGE	0	1	1	if out ≥ 0 jump
JLT	1	0	0	if out < 0 jump
JNE	1	0	1	if out ≠ 0 jump
JLE	1	1	0	if out ≤ 0 jump
JMP	1	1	1	Unconditional jump

End